
SEMINARIO ISGI (S1) ENTORNO



AGENDA



- Introducción
- Lenguaje y entorno de desarrollo
- Repositorio
- Configuración del entorno y primer proyecto
- Notas adicionales
- Qué hace y qué no OpenGL
- Tubería gráfica en OpenGL
- Librerías relacionadas con OpenGL
- Sintaxis en OpenGL y GLUT
- Código ejemplo
- Compilar Hola_Mundo
- Nuevas funciones y constantes

INTRODUCCIÓN

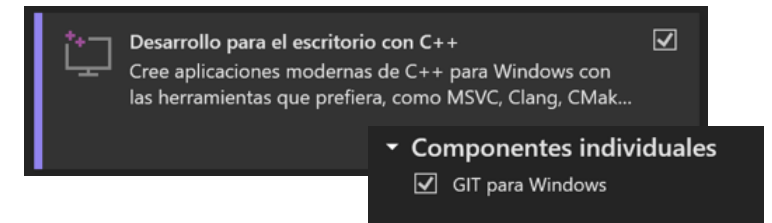
- Objetivos:
 - Instalación y puesta en marcha del entorno de desarrollo de software gráfico
 - Completar la cadena de compilación, montaje y ejecución



LENGUAJE Y ENTORNO DE DESARROLLO



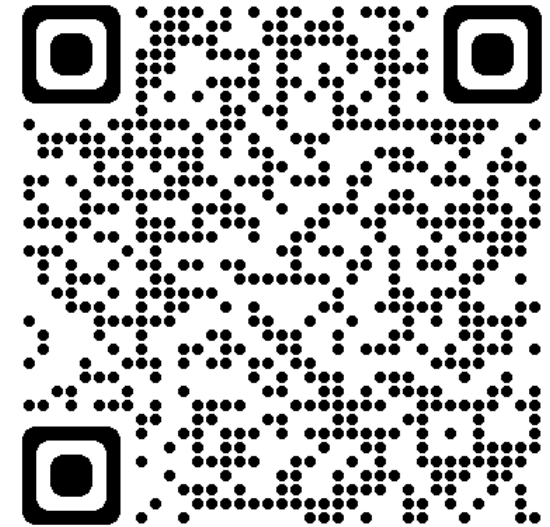
- Se utiliza C++ en el entorno de desarrollo Visual C++ de Microsoft
 - Los laboratorios ya tienen instalado MS Visual Studio
- Para trabajar en nuestro propio computador Windows, debemos instalar Visual Studio
 - Descargar el instalador de Visual Studio Community 202x desde la página de Microsoft
<https://visualstudio.microsoft.com/es/vs/community>
 - Seleccionar las opciones en el instalador
 - C++ para escritorio
 - Extensiones para git
- Para trabajar en nuestro propio computador NO Windows
 - <http://polilabsvpn.upv.es> -> DSIC-GPU
 - conectarse por vpn fuera del poli



REPOSITORIO

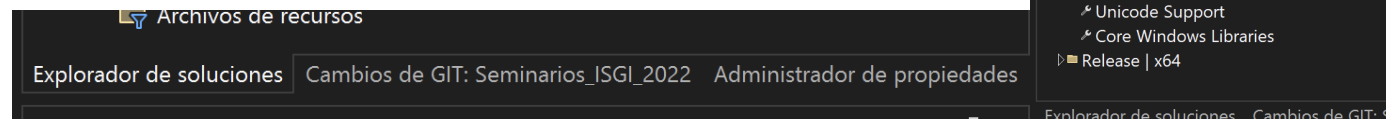
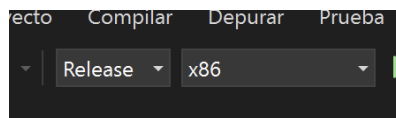
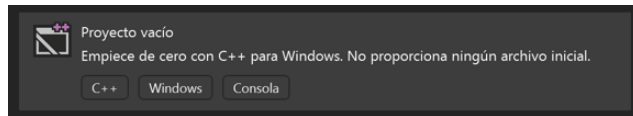
- El código necesario para trabajar en esta asignatura se encuentra disponible en el repositorio:

https://github.com/RobVivo/Practicas_GL



CONFIGURACIÓN DEL ENTORNO Y PRIMER PROYECTO

- Clonar el repositorio en nuestro disco local
- Indicar dónde se encuentran las bibliotecas dinámicas
- Abrir el fichero con extensión 'sln' en Visual Studio
- Crear un nuevo proyecto y agregarlo a la solución
- Cambiar las propiedades del proyecto para reconocer los directorios de cabeceras y bibliotecas



NOTAS ADICIONALES



- Leer la documentación y familiarizarse con el entorno
 - Una fuente de consulta útil sobre Visual C++ es el MSDN de Microsoft
 - Sobre asuntos del lenguaje C++ y librerías asociadas puede consultarse también
- Es interesante usar la Standard Library para C++
 - Para el manejo de entrada/salida, strings, templates para estructuras de datos dinámicas, y las librerías clásicas de C.
- Es importante incluir los ficheros cabecera de la Standard Library (p.e. `cstdio`) antes que `freeglut.h`

QUÉ HACE Y QUÉ NO OPENGL



OpenGL es una especificación de la interfaz entre la aplicación de gráficos 3D y el hardware gráfico (API)

- Consiste en una máquina de estados y algoritmos de procesamiento gráfico

Realiza todo el procesamiento gráfico usando el hardware existente

- Representa en pantalla primitivas gráficas definidas en el espacio 3D

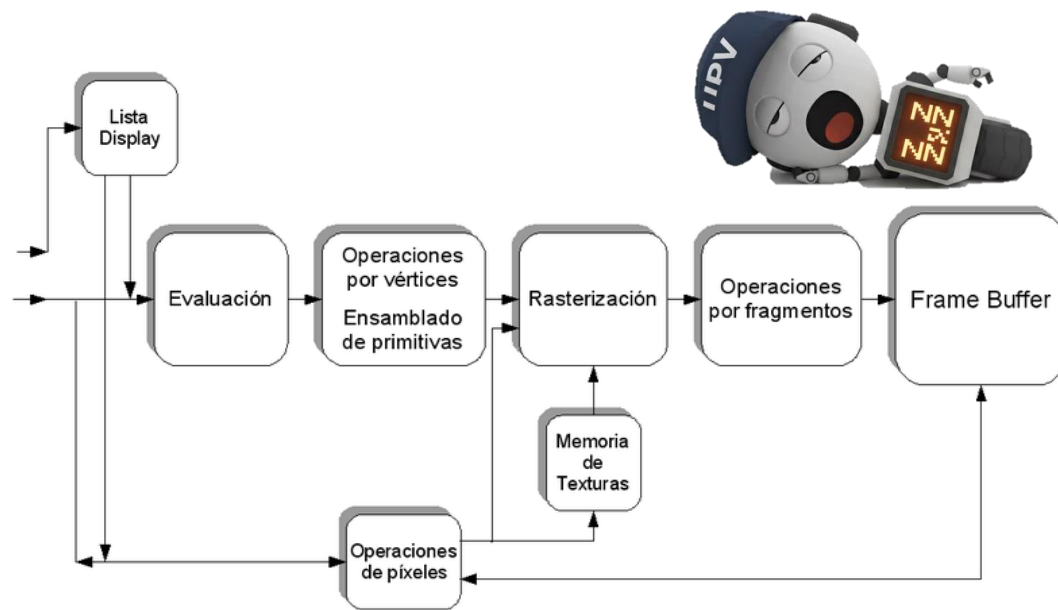
Es independiente del hardware y la plataforma usada

- No incluye funciones para gestionar ventanas ni eventos de usuario
- Estas tareas se encomiendan a librerías construidas encima de OpenGL

No proporciona funciones para la creación, gestión o interrelación de objetos gráficos 3D

- Se recomiendan herramientas construidas sobre OpenGL como OpenSceneGraph

TUBERÍA GRÁFICA EN OPENGL



Entidades
gráficas
principales:
vértices y
fragmentos

- Los vértices hacen referencia a un punto en 3D y sus atributos
- Los fragmentos son elementos pre-píxel, con información de color, profundidad y transparencia

Operaciones por
vértice y
fragmento en la
tubería fija

- Siempre son las mismas y se aplican dependiendo de si están habilitadas o deshabilitadas

Posibilidad de
programar el
comportamiento
de cada fase
mediante
“shaders”

- Este tipo de tubería se denomina dinámica



BIBLIOTECAS RELACIONADAS CON OPENGL

- OpenGL
 - Incluye gl.h, glu.h, opengl32.lib y opengl32.dll
 - Se instala con el entorno de desarrollo
- GLUT (freeglut)
 - Incluye freeglut.h, freeglut32.lib y freeglut32.dll
 - Se recomienda usar freeglut
- GLEW
 - Envoltorio para gl.h para usar extensiones más allá de la versión 1.2 de OpenGL
 - Ofrece dos bibliotecas: glew32.lib y glew32s.lib
- GLUI
 - Ofrece controles avanzados contruidos sobre OpenGL





SINTAXIS EN OPENGL Y GLUT



Funciones con prefijos `gl`, `glu`, `glut`

El resto del nombre sigue el convenio de iniciales



Tipos predefinidos

Por ejemplo, `GLfloat`



Constantes con prefijos `GL_` o `GLUT_`

El nombre es en mayúsculas y separado por guiones bajos



Orientado al lenguaje C

No hay polimorfismo en las funciones

Se usa un sufijo diferente para la misma función con diferentes parámetros

CÓDIGO DE EJEMPLO

```
#include <iostream>
#include <gl\freeglut.h>
void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glFlush();
}
void reshape(GLint w, GLint h)
{
}
void main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
    glutInitWindowSize(200,200);
    glutCreateWindow("Hola Mundo");
    std::cout << "Hola Mundo running" << std::endl;
    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutMainLoop();
}
```



COMPILAR HOLA MUNDO



```
#include <iostream>
#include <gl\freeglut.h>
void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glFlush();
}
void reshape(GLint w, GLint h)
{
}
void main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
    glutInitWindowSize(200,200);
    glutCreateWindow("Hola Mundo");
    std::cout << "Hola Mundo running" << std::endl;
    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutMainLoop();
}
```

El código fuente se implementa en el archivo agregado a 'Archivos de origen'

- Se incluyen las bibliotecas necesarias
- Se definen las funciones display() y reshape(int,int)
- Se registra en GLUT antes del inicio del bucle de atención a eventos

Para generar el ejecutable, se usa la opción del menú 'Compilar->Compilar Hola_Mundo'

- La ejecución produce dos ventanas: la consola y la ventana de dibujo

Las funciones display() y reshape(int,int) son callbacks

- Se ejecutan cuando se dispara un evento de esa clase
- El manejador de eventos 'MainLoop' recoge los eventos de usuario y llama a la función correspondiente

NUEVAS FUNCIONES Y CONSTANTES: OPENGL Y GLUT

- `glClear()`: Clears any buffer
- `glFlush()`: dumps all pending drawing orders
- `GL_COLOR_BUFFER_BIT`: The rendered image
- `glutInit()`: Initializes GLUT and must be called before any other GLUT call.
- `glutInitDisplayMode(bitmask)`
 - Default configuration is a color buffer without alpha channel in RGB mode.
 - Parameter is a bitmask by or combination of predefined constants.
- `glutInitWindowSize(int ancho, int alto)`: Sets the size of the window
- `int glutCreateWindow(char* titulo)`: Creates the window, without showing it yet, and configures all its characteristics such as size, position and buffer format.
- `glutDisplayFunc()`: Allows to register the function that will attend the events





Chat Rufus SGI

* Se precisa cuenta ChatGPT